

在 Cloud Dataproc 使用 Apache Spark 和 Jupyter 筆記本

來源：<https://codelabs.developers.google.com/codelabs/spark-jupyter-dataproc?hl=zh-tw>

步驟數：10

本研究室說明如何使用選用元件和元件閘道，在 Cloud Dataproc 上設定 Apache Spark 和 Jupyter Notebook。

目錄

1. [總覽](#)
2. [建立專案](#)
3. [設定環境](#)
4. [建立 GCS bucket](#)
5. [建立包含 Jupyter 和元件閘道的 Dataproc 叢集](#)
6. [建立 Apache Spark 筆記本](#)
7. [在筆記本中使用 Python 繪圖程式庫](#)
8. [最佳化提示 - 將資料快取在記憶體中](#)
9. [更多用途的範例筆記本](#)
10. [清除所用資源](#)

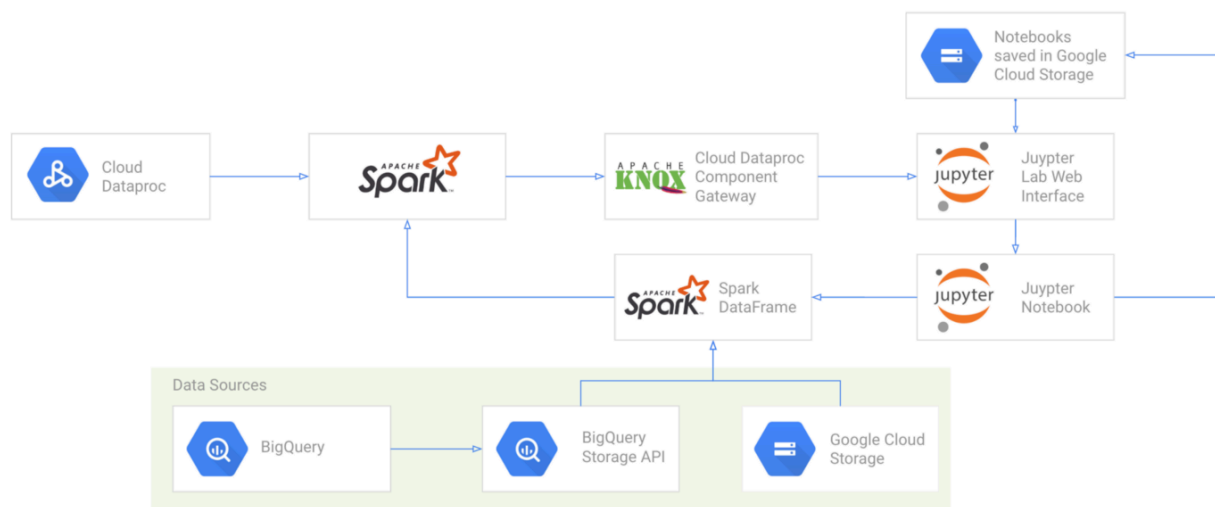
步驟 1 · 約 0 分鐘

1. 總覽

本實驗室將說明如何在 Cloud Dataproc 上設定及使用 [Apache Spark](#) 和 [Jupyter 筆記本](#)。

Jupyter 記事本可讓您以互動方式執行程式碼，並立即查看結果，因此廣泛用於探索性資料分析和建構機器學習模型。

不過，設定及使用 Apache Spark 和 Jupyter 筆記本可能很複雜。



Cloud Dataproc 可讓您在約 90 秒內建立包含 Apache Spark、[Jupyter 元件](#)和[元件閘道](#)的 Dataproc 叢集，輕鬆快速地完成這項工作。

課程內容

在本程式碼研究室中，您可以瞭解如何執行以下操作：

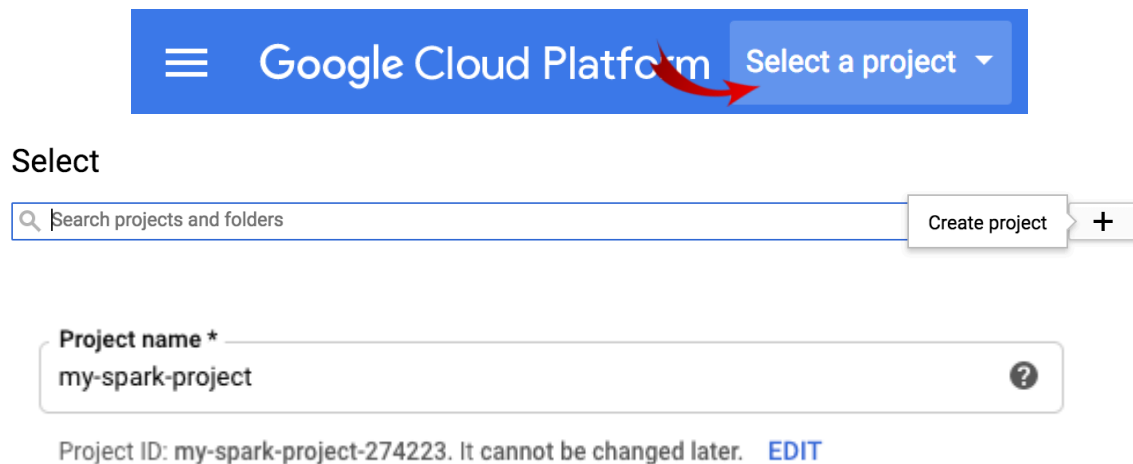
- 為叢集建立 Google Cloud Storage bucket
- 建立包含 Jupyter 和元件閘道的 Dataproc 叢集，
- 存取 Dataproc 上的 JupyterLab 網頁介面
- 建立筆記本，並使用 Spark BigQuery Storage 連接器
- 執行 Spark 工作並繪製結果。

在 Google Cloud 上執行這個實驗室的總費用約為 \$1 美元。如要瞭解 Cloud Dataproc 的完整定價資訊，請參閱[這篇文章](#)。

步驟 2 · 約 2 分鐘

2. 建立專案

前往 console.cloud.google.com 登入 Google Cloud Platform Console，然後建立新專案：



Google Cloud Platform Select a project ▾

Select

Search projects and folders Create project +

Project name *
my-spark-project ?

Project ID: my-spark-project-274223. It cannot be changed later. [EDIT](#)

接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Google Cloud 資源。

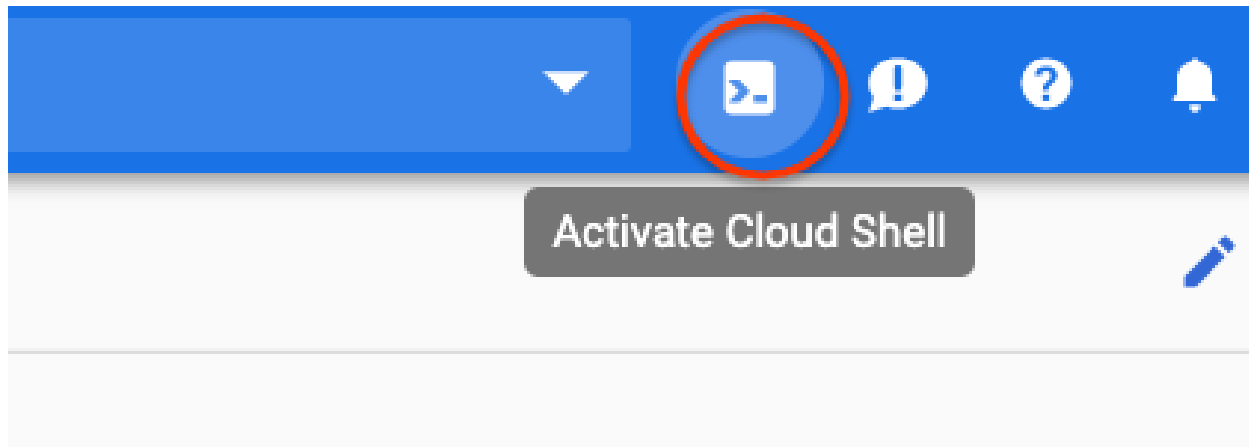
完成這項程式碼研究室的費用不應超過數美元，但如果您決定使用更多資源，或是將資源繼續執行，則可能會增加費用。本程式碼研究室的最後一節將逐步說明如何清除專案。

Google Cloud Platform 新使用者享有價值 [\\$300 美元的免費試用期](#)。

步驟 3 · 約 3 分鐘

3. 設定環境

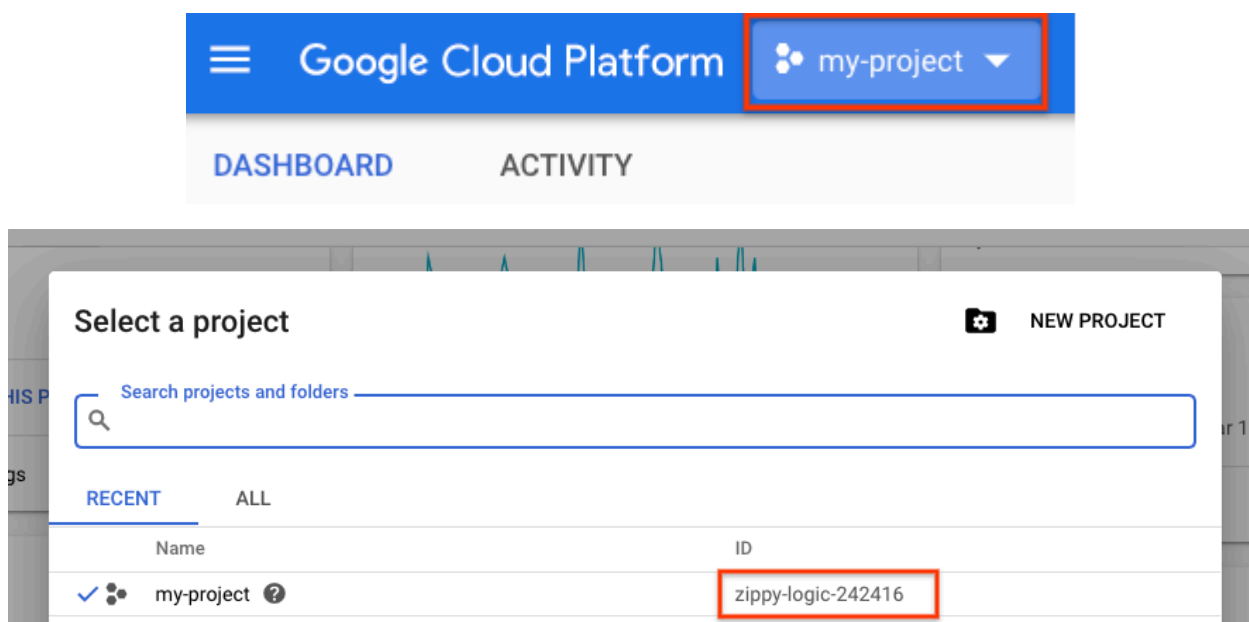
首先，按一下雲端控制台右上角的按鈕，開啟 Cloud Shell：



Cloud Shell 載入後，請執行下列指令，設定上一個步驟中的專案 ID：

```
gcloud config set project <project_id>
```

您也可以在 Cloud 控制台的左上方點選專案，即可找到專案 ID：



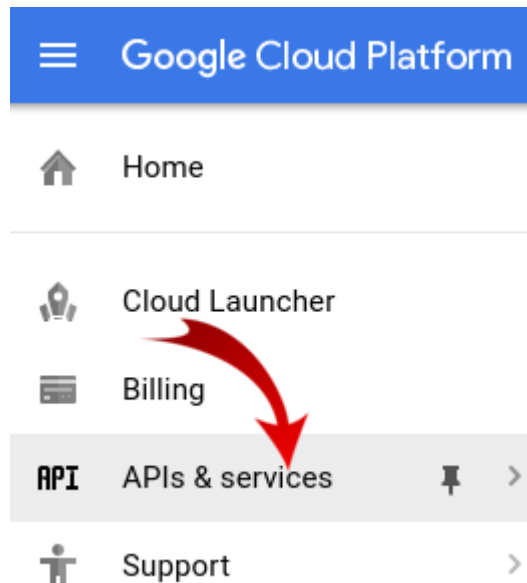
接著，請啟用 Dataproc、Compute Engine 和 BigQuery Storage API。

```
gcloud services enable dataproc.googleapis.com \  
compute.googleapis.com \  
storage-component.googleapis.com \  
bigquery.googleapis.com \  
bigquerystorage.googleapis.com
```

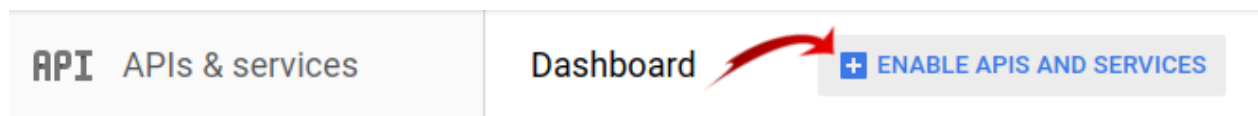
您也可以在 [Cloud 控制台](#) 中執行這項操作。按一下畫面左上方的「選單」圖示。



從下拉式選單中選取「API 管理工具」。



按一下「啟用 API 和服務」。



搜尋並啟用下列 API：

- Compute Engine API
- Dataproc API
- BigQuery API
- BigQuery Storage API

步驟 4 · 約 2 分鐘

4. 建立 GCS bucket

在最靠近資料的區域建立 Google Cloud Storage bucket，並為其命名。

這會用於 Dataproc 叢集。

```
REGION=us-central1
BUCKET_NAME=<your-bucket-name>

gsutil mb -c standard -l ${REGION} gs://${BUCKET_NAME}
```

輸出內容應如下所示：

```
Creating gs://<your-bucket-name>/...
```

步驟 5 · 約 5 分鐘

5. 建立包含 Jupyter 和元件閘道的 Dataproc 叢集

建立叢集

設定叢集的環境變數

```
REGION=us-central1
ZONE=us-central1-a
CLUSTER_NAME=spark-jupyter
BUCKET_NAME=<your-bucket-name>
```

然後執行這項 gcloud 指令，建立叢集並搭載所有必要元件，以便在叢集上使用 Jupyter。

```
gcloud beta dataproc clusters create ${CLUSTER_NAME} \
  --region=${REGION} \
  --image-version=1.4 \
  --master-machine-type=n1-standard-4 \
  --worker-machine-type=n1-standard-4 \
  --bucket=${BUCKET_NAME} \
  --optional-components=ANACONDA,JUPYTER \
  --enable-component-gateway
```

叢集建立期間，您應該會看到下列輸出內容

```
Waiting on operation [projects/spark-jupyter/regions/us-central1/operations/abcd123456].
Waiting for cluster creation operation...
```

建立叢集大約需要 90 秒，完成後即可透過 [Dataproc Cloud 控制台 UI](#) 存取叢集。

等待期間，您可以繼續閱讀下文，進一步瞭解 gcloud 指令中使用的標記。

叢集建立完成後，您應該會看到下列輸出內容：

```
Created [https://dataproc.googleapis.com/v1beta2/projects/project-id/regions/us-central1/clusters/spark-jupyter] Cluster placed in zone [us-central1-a].
```

gcloud dataproc create 指令中使用的旗標

以下是 gcloud Dataproc create 指令中使用的旗標明細

```
--region=${REGION}
```

指定要建立叢集的區域和可用區。如要查看適用區域清單，請按[這裡](#)。

```
--image-version=1.4
```

要在叢集中使用的映像檔版本。如要查看可用版本清單，請按[這裡](#)。

```
--bucket=${BUCKET_NAME}
```

指定您先前建立的 Google Cloud Storage bucket，供叢集使用。如未提供 GCS bucket，系統會為您建立。

即使刪除叢集，筆記本也會儲存在這裡，因為 GCS 值區不會遭到刪除。

```
--master-machine-type=n1-standard-4  
--worker-machine-type=n1-standard-4
```

用於 Dataproc 叢集的機器類型。如要查看可用的機器類型清單，請按[這裡](#)。

如未設定 -num-workers 旗標，系統預設會建立 1 個主要節點和 2 個工作站節點

```
--optional-components=ANACONDA,JUPYTER
```

為[選用元件](#)設定這些值，即可在叢集上安裝 Jupyter 和 Anaconda (Jupyter 筆記本需要 Anaconda) 的所有必要程式庫。

```
--enable-component-gateway
```

啟用「元件閘道」會使用 Apache Knox 和反向 Proxy 建立 App Engine 連結，讓您輕鬆、安全地存取 Jupyter 和 JupyterLab 網頁介面，且不必再建立 SSH 通道。

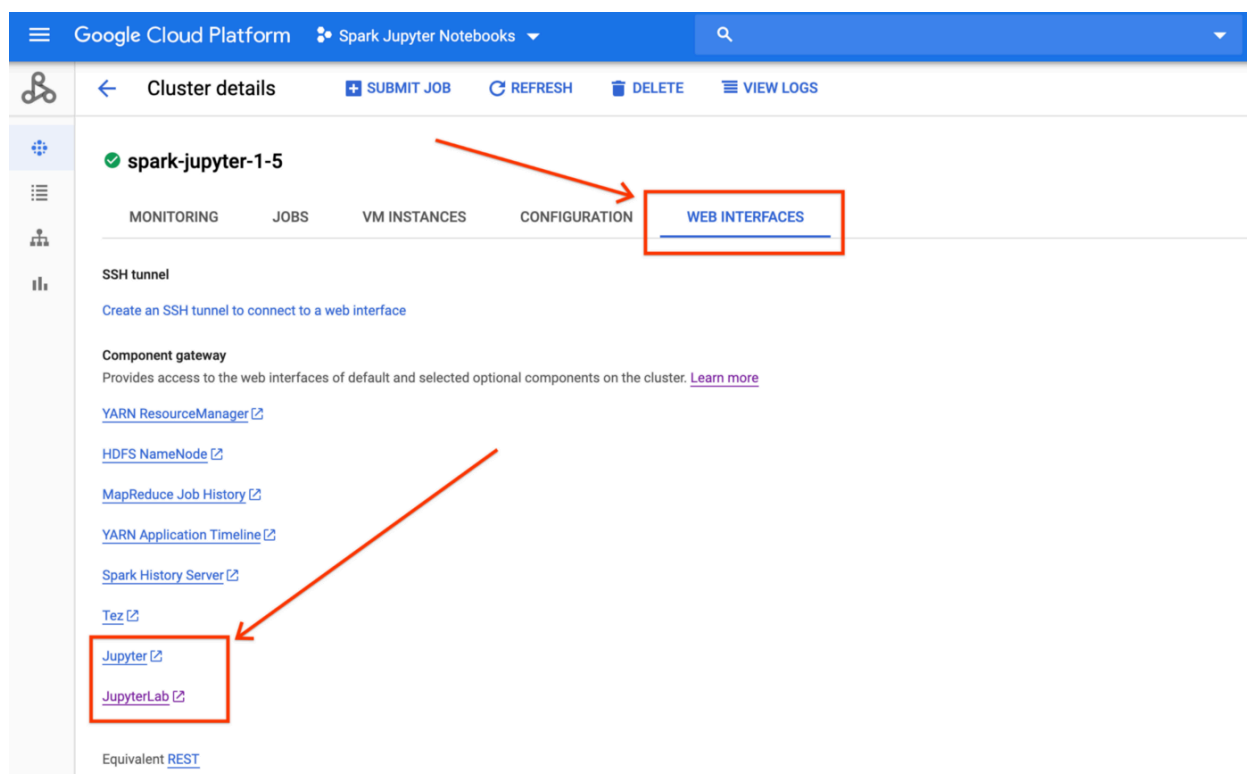
此外，系統也會為叢集上的其他工具建立連結，包括 Yarn 資源管理員和 Spark 記錄伺服器，方便您查看作業的效能和叢集使用模式。

步驟 6 · 約 10 分鐘

6. 建立 Apache Spark 筆記本

存取 JupyterLab 網頁介面

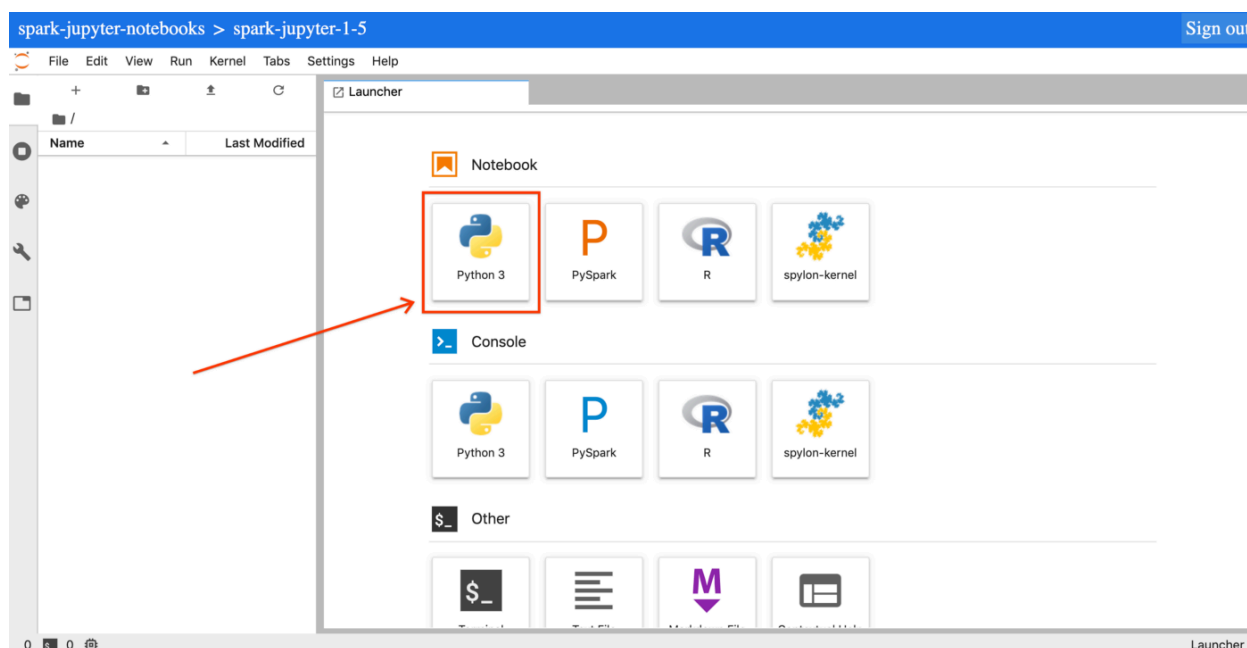
叢集就緒後，請前往 [Dataproc 叢集 - Cloud 控制台](#)，按一下您建立的叢集，然後前往「Web Interfaces」(網頁介面) 分頁，即可找到 JupyterLab 網頁介面的元件閘道連結。



您會發現自己可以存取 Jupyter (傳統筆記本介面) 或 JupyterLab (Project Jupyter 的新一代 UI)。

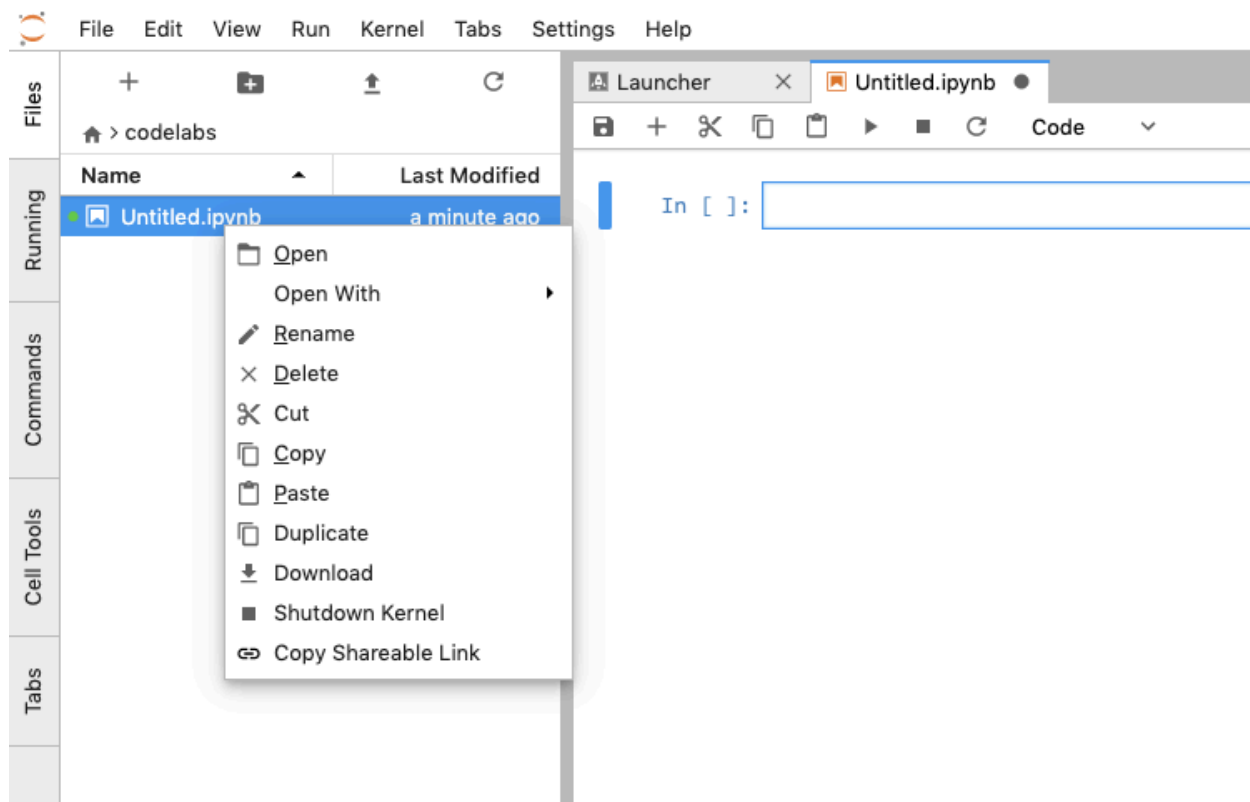
JupyterLab 提供了許多實用的全新 UI 功能，因此如果您是筆記本新手，或是想使用最新改良功能，建議使用 JupyterLab，因為根據官方文件，JupyterLab 最終會取代傳統的 Jupyter 介面。

建立以 Python 3 核心為基礎的筆記本



在啟動器分頁中，按一下 Python 3 筆記本圖示，建立具有 Python 3 核心 (而非 PySpark 核心) 的筆記本，即可在筆記本中設定 SparkSession，並加入使用 [BigQuery Storage API](#) 時所需的 [spark-bigquery-connector](#)。

重新命名筆記本



在左側邊欄或頂端導覽列中，按一下筆記本名稱，然後將筆記本重新命名為「BigQuery Storage & Spark DataFrames.ipynb」

在筆記本中執行 Spark 程式碼

```
In [2]: !scala -version
cat: /release: No such file or directory
Scala code runner version 2.11.12 -- Copyright 2002-2017, LAMP/EPFL

Create Spark Session

Include the correct version of the spark-bigquery-connector jar

Scala version 2.11 - 'gs://spark-lib/bigquery/spark-bigquery-latest.jar'.
Scala version 2.12 - 'gs://spark-lib/bigquery/spark-bigquery-latest_2.12.jar'.

In [3]: from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName('1.1. BigQuery Storage & Spark DataFrames - Python') \
    .config('spark.jars', 'gs://spark-lib/bigquery/spark-bigquery-latest.jar') \
    .getOrCreate()

Enable repl.eagerEval

This will output the results of DataFrames in each step without the need to show df.show() and also improves the formatting of the output

In [4]: spark.conf.set("spark.sql.repl.eagerEval.enabled", True)

Read BigQuery table into Spark DataFrame

Use filter() to query data from a partitioned table.

In [5]: table = "bigquery-public-data.wikipedia.pageviews_2020"
df_wiki_pageviews = spark.read \
    .format("bigquery") \
    .option("table", table) \
    .option("filter", "datehour >= '2020-03-01' AND datehour < '2020-03-02'" ) \
    .load()

df_wiki_pageviews.printSchema()

root
 |-- datehour: timestamp (nullable = true)
 |-- wiki: string (nullable = true)
 |-- title: string (nullable = true)
 |-- views: long (nullable = true)

Select required columns and apply a filter using where() which is an alias for filter() then cache the table

In [7]: df_wiki_en = df_wiki_pageviews \
    .select("title", "wiki", "views") \
    .where("views > 1000 AND wiki in ('en', 'en.m')") \
    .cache()

df_wiki_en

Out[7]:
```

	title	wiki	views
2020_Democratic_P...	en	3242	
Eurovision_Song_C...	en	2368	
Collin_McRae	en	2360	
Donald_trump	en	2223	
Comparison_of_onl...	en	1398	

在本筆記本中，您將使用 [spark-bigquery-connector](#)，這是用於在 BigQuery 和 Spark 之間讀取及寫入資料的工具，可運用 [BigQuery Storage API](#)。

BigQuery Storage API 採用 [RPC 架構的通訊協定](#)，可大幅提升 BigQuery 資料存取效能。支援平行讀取和寫入資料，以及各種序列化格式，例如 [Apache Avro](#) 和 [Apache Arrow](#)。整體而言，這代表效能大幅提升，尤其是在較大的資料集上。

在第一個儲存格中檢查叢集的 Scala 版本，以便加入正確版本的 spark-bigquery-connector jar。

輸入 [1]:

```
!scala -version
```

輸出 [1]:

```
Scala code runner version 2.11.12 -- Copyright 2002-2017, LAMP/EPFL
```

建立 Spark 工作階段，並加入 spark-bigquery-connector 套件。

如果 Scala 版本為 2.11，請使用下列套件。

```
com.google.cloud.spark:spark-bigquery-with-dependencies_2.11:0.15.1-beta
```

如果 Scala 版本為 2.12，請使用下列套件。

```
com.google.cloud.spark:spark-bigquery-with-dependencies_2.12:0.15.1-beta
```

輸入 [2] :

```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName('BigQuery Storage & Spark DataFrames') \
    .config('spark.jars.packages', 'com.google.cloud.spark:spark-bigquery-with-
dependencies_2.11:0.15.1-beta') \
    .getOrCreate()
```

啟用 repl.eagerEval

這樣一來，系統就會輸出每個步驟的 DataFrame 結果，不必再顯示 df.show()，輸出格式也會更完善。

輸入 [3] :

```
spark.conf.set("spark.sql.repl.eagerEval.enabled",True)
```

將 BigQuery 資料表讀取至 Spark DataFrame

從公開 BigQuery 資料集讀取資料，建立 Spark DataFrame。這會使用 [spark-bigquery-connector](#) 和 BigQuery Storage API，將資料載入 Spark 叢集。

建立 Spark DataFrame，並從 [BigQuery 公開資料集 \(Wikipedia 網頁瀏覽量\)](#) 載入資料。您會發現自己並未對資料執行查詢，因為您是使用 [spark-bigquery-connector](#) 將資料載入 Spark，並在其中處理資料。執行這段程式碼時，系統不會實際載入資料表，因為這是 Spark 中的延遲評估，執行作業會在下一個步驟中進行。

輸入內容 [4] :

```
table = "bigquery-public-data.wikipedia.pageviews_2020"

df_wiki_pageviews = spark.read \
    .format("bigquery") \
    .option("table", table) \
    .option("filter", "datehour >= '2020-03-01' AND datehour < '2020-03-02'") \
    .load()

df_wiki_pageviews.printSchema()
```

輸出內容 [4] :

```
root
|-- datehour: timestamp (nullable = true)
|-- wiki: string (nullable = true)
|-- title: string (nullable = true)
|-- views: long (nullable = true)
```

選取必要欄，然後使用 `where()` (`filter()` 的別名) 套用篩選器。

執行這段程式碼時，系統會觸發 Spark 動作，並從 BigQuery Storage 讀取資料。

輸入 [5] :

```
df_wiki_en = df_wiki_pageviews \
    .select("datehour", "wiki", "views") \
    .where("views > 1000 AND wiki in ('en', 'en.m')") \

df_wiki_en
```

輸出內容 [5] :

datehour	wiki	views
2020-03-01 23:00:00	en	3242
2020-03-01 22:00:00	en	2368
2020-03-01 22:00:00	en	2360
2020-03-01 22:00:00	en	2223
2020-03-01 22:00:00	en	1398
2020-03-01 23:00:00	en	1872
2020-03-01 23:00:00	en	136620
2020-03-01 23:00:00	en	1084
2020-03-01 23:00:00	en	1946
2020-03-01 23:00:00	en	8313
2020-03-01 23:00:00	en	1084
2020-03-01 22:00:00	en	3524
2020-03-01 22:00:00	en	1328
2020-03-01 22:00:00	en	1297
2020-03-01 23:00:00	en	1968
2020-03-01 22:00:00	en	1139
2020-03-01 22:00:00	en	1006
2020-03-01 23:00:00	en	1511
2020-03-01 23:00:00	en	1526
2020-03-01 22:00:00	en	1405

only showing top 20 rows

依標題分組依據並依網頁瀏覽次數排序，即可查看熱門網頁

輸入 [6] :

```
import pyspark.sql.functions as F

df_datehour_totals = df_wiki_en \
    .groupBy("datehour") \
    .agg(F.sum('views').alias('total_views'))

df_datehour_totals.orderBy('total_views', ascending=False)
```

輸出內容 [6] :

datehour	total_views
2020-03-01 21:00:00	1642981
2020-03-01 06:00:00	1591160
2020-03-01 22:00:00	1541455
2020-03-01 17:00:00	1535983
2020-03-01 18:00:00	1495387
2020-03-01 16:00:00	1487786
2020-03-01 05:00:00	1469068
2020-03-01 07:00:00	1458756
2020-03-01 20:00:00	1457051
2020-03-01 15:00:00	1446984
2020-03-01 19:00:00	1427811
2020-03-01 14:00:00	1372760
2020-03-01 23:00:00	1353548
2020-03-01 08:00:00	1353292
2020-03-01 03:00:00	1339853
2020-03-01 04:00:00	1312186
2020-03-01 12:00:00	1225647
2020-03-01 13:00:00	1212003
2020-03-01 10:00:00	1211310
2020-03-01 09:00:00	1200977

only showing top 20 rows

步驟 7 · 約 5 分鐘

7. 在筆記本中使用 Python 繪圖程式庫

您可以利用 Python 中提供的各種繪圖程式庫，繪製 Spark 作業的輸出內容。

將 Spark DataFrame 轉換為 Pandas DataFrame

將 Spark DataFrame 轉換為 Pandas DataFrame，並將 datehour 設為索引。如果您想直接在 Python 中處理資料，並使用許多可用的 Python 繪圖程式庫繪製資料，這項功能就非常實用。

輸入 [7]：

```
spark.conf.set("spark.sql.execution.arrow.enabled", "true")
pandas_datehour_totals = df_datehour_totals.toPandas()

pandas_datehour_totals.set_index('datehour', inplace=True)
pandas_datehour_totals.head()
```

輸出內容 [7]：

	total_views
datehour	
2020-03-01 22:00:00	1541455
2020-03-01 09:00:00	1200977
2020-03-01 12:00:00	1225647
2020-03-01 20:00:00	1457051
2020-03-01 10:00:00	1211310

繪製 Pandas DataFrame 圖表

匯入 matplotlib 程式庫，這是筆記本中顯示繪圖時的必要程式庫

輸入 [8]：

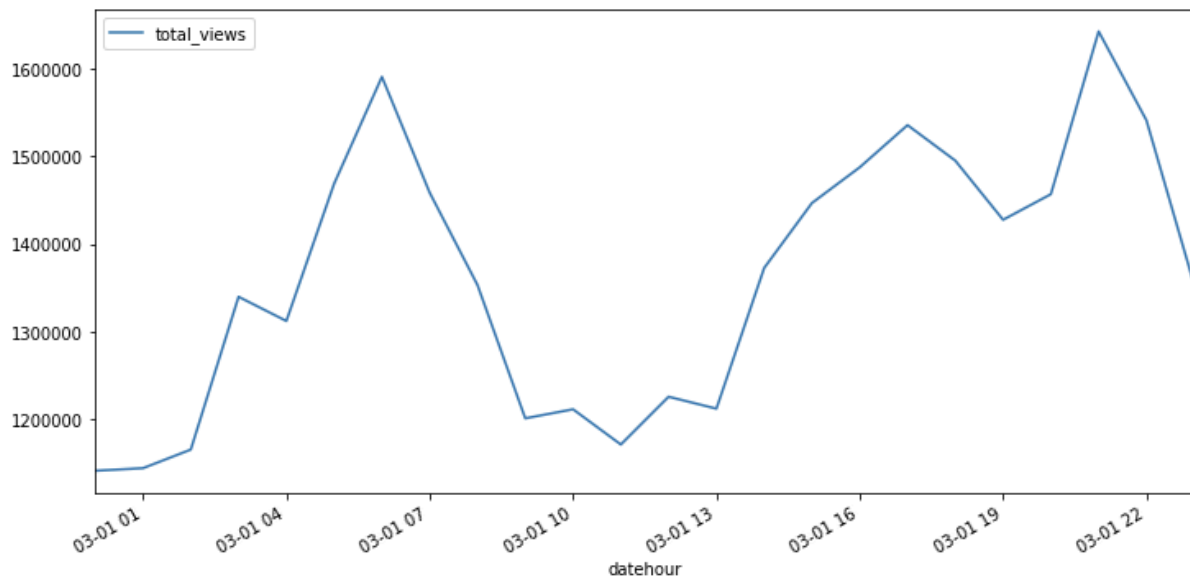
```
import matplotlib.pyplot as plt
```

使用 [Pandas 繪圖函式](#)，從 Pandas DataFrame 建立折線圖。

輸入 [9]：

```
pandas_datehour_totals.plot(kind='line',figsize=(12,6));
```

輸出內容 [9]：



確認筆記本已儲存至 GCS

現在，您應該已在 Dataproc 叢集上執行第一個 Jupyter 筆記本。為筆記本命名，系統就會自動將筆記本儲存至建立叢集時使用的 GCS bucket。

您可以在 Cloud Shell 中使用下列 gsutil 指令檢查這項設定：

```
BUCKET_NAME=<your-bucket-name>  
gsutil ls gs://{BUCKET_NAME}/notebooks/jupyter
```

輸出內容應如下所示：

```
gs://bucket-name/notebooks/jupyter/  
gs://bucket-name/notebooks/jupyter/BigQuery Storage & Spark DataFrames.ipynb
```

8. 最佳化提示 - 將資料快取在記憶體中

在某些情況下，您可能希望資料位於記憶體中，而不是每次都從 BigQuery 儲存空間讀取。

這項工作會從 BigQuery 讀取資料，並將篩選條件推送至 BigQuery。接著，系統會在 Apache Spark 中計算匯總資料。

```
import pyspark.sql.functions as F

table = "bigquery-public-data.wikipedia.pageviews_2020"

df_wiki_pageviews = spark.read \
    .format("bigquery") \
    .option("table", table) \
    .option("filter", "datehour >= '2020-03-01' AND datehour < '2020-03-02'") \
    .load()

df_wiki_en = df_wiki_pageviews \
    .select("title", "wiki", "views") \
    .where("views > 10 AND wiki in ('en', 'en.m')")

df_wiki_en_totals = df_wiki_en \
    .groupBy("title") \
    .agg(F.sum('views').alias('total_views'))

df_wiki_en_totals.orderBy('total_views', ascending=False)
```

您可以修改上述工作，加入表格的快取，現在 Apache Spark 會在記憶體中套用 wiki 資料欄的篩選器。

```

import pyspark.sql.functions as F

table = "bigquery-public-data.wikipedia.pageviews_2020"

df_wiki_pageviews = spark.read \
    .format("bigquery") \
    .option("table", table) \
    .option("filter", "datehour >= '2020-03-01' AND datehour < '2020-03-02'") \
    .load()

df_wiki_all = df_wiki_pageviews \
    .select("title", "wiki", "views") \
    .where("views > 10")

# cache the data in memory
df_wiki_all.cache()

df_wiki_en = df_wiki_all \
    .where("wiki in ('en', 'en.m')")

df_wiki_en_totals = df_wiki_en \
    .groupBy("title") \
    .agg(F.sum('views').alias('total_views'))

df_wiki_en_totals.orderBy('total_views', ascending=False)

```

接著，您可以使用快取資料篩選其他維基語言，不必再次從 BigQuery 儲存空間讀取資料，因此執行速度會快上許多。

```

df_wiki_de = df_wiki_all \
    .where("wiki in ('de', 'de.m')")

df_wiki_de_totals = df_wiki_de \
    .groupBy("title") \
    .agg(F.sum('views').alias('total_views'))

df_wiki_de_totals.orderBy('total_views', ascending=False)

```

您可以執行下列指令來移除快取：

```
df_wiki_all.unpersist()
```

9. 更多用途的範例筆記本

[Cloud Dataproc GitHub 存放區](#)提供 Jupyter 筆記本，其中包含常見的 Apache Spark 模式，可搭配各種 Google Cloud Platform 產品和開放原始碼工具載入、儲存及繪製資料：

- [1.1. BigQuery Storage 和 Spark DataFrames](#)
 - [1.2. BigQuery Storage 和 Spark SQL](#)
 - [1.3. BigQuery Storage 和 Spark MLlib](#)
 - [2.1. Google Cloud Storage \(CSV\) 和 Spark DataFrame](#)
 - [3.1. Spark DataFrame 和 Pandas 繪圖](#)
-

10. 清除所用資源

完成本快速入門導覽課程後，如要避免系統向您的 GCP 帳戶收取不必要的費用，請按照下列步驟操作：

1. 刪除您建立的環境 [Cloud Storage bucket](#)
2. 刪除 [Dataproc 環境](#)。

如果您專為本程式碼研究室建立了專案，也可以選擇刪除該專案：

1. 前往 GCP 主控台的「Projects」(專案) 頁面。
2. 在專案清單中選取要刪除的專案，然後點按「Delete」(刪除)。
3. 在方塊中輸入專案 ID，然後按一下「Shut down」(關閉) 刪除專案。

警告：刪除專案會出現以下結果：

- **專案中的所有內容都會遭到刪除。**如果您之前使用現有的專案來進行本教學課程，當您刪除該專案時，也會一併刪除您在該專案中完成的所有其他工作。
- **自訂專案 ID 不復存在。**您建立這個專案時，可能已建立日後使用的自訂專案 ID。如要保留使用該專案 ID 的網址 (例如 [appspot.com](#) 網址)，請刪除在該專案中選取的資源，而不是刪除整個專案。

授權

這項內容採用的授權為創用 CC 姓名標示 3.0 通用授權和 Apache 2.0 授權。
